Université Paris-Dauphine – PSL

Master 2 IASD apprentissage

Comparaison d'architectures de réseaux de neurones pour la prédiction dans le jeu de Go 19×19

Sous contrainte de 100 000 paramètres

Rapport de projet – Apprentissage Profond

Amine ROUIBI - Thomas SINAPI - Mohamed ZOUAD

Année universitaire 2025–2026
1er avril 2026

Résumé

Ce rapport présente une étude comparative de quatorze architectures de réseaux de neurones profonds avec les mêmes hyper-paramètres, appliquées à la prédiction dans le jeu de Go 19×19. La contrainte principale imposée est un budget de **100 000 paramètres maximum**. La tâche consiste en une prédiction à deux têtes : une tête *politique* estimant la probabilité de chaque coup légal (361 sorties), et une tête *valeur* évaluant les chances de victoire du joueur courant. L'étude s'organise en trois phases : une exploration initiale de dix familles architecturales (Phase 1), un entraînement approfondi des trois meilleures avec davantage de données (Phase 2), et l'évaluation de quatre architectures efficientes modernes inspirées des avancées MobileNet/ConvNeXt/EfficientFormer (Phase 3). Le meilleur modèle atteint **44,56 %** de précision politique en validation, soit un gain de plus de 43 points de pourcentage par rapport à la base MLP.

Table des matières

1 Introduction **3**

2 Cadre expérimental **3**
2.1 Représentation de l'entrée 3
2.2 Sorties et fonctions de perte 3
2.3 Protocole d'entraînement 4
2.4 Données d'entraînement 4
2.5 Environnement de calcul et outils 5

3 Phase 1 : Exploration de dix architectures **5**
3.1 Architectures explorées 5
3.2 Résultats de la Phase 1 6
3.3 Courbes d'entraînement 7
3.4 Analyse 8

4 Phase 2 : Entraînement approfondi des meilleures architectures **8**
4.1 Motivation et sélection 8
4.2 Résultats 9
4.3 Courbes d'entraînement 9
4.4 Analyse 10

5 Phase 3 : Architectures efficientes modernes **10**
5.1 Motivation 10
5.2 Architectures de la Phase 3 10
5.3 Résultats 11
5.4 Analyse 11

6 Comparaison globale **12**
6.1 Discussion 13

7 Conclusion **14**

1 Introduction

Le jeu de Go sur un plateau 19×19 est l'un des problèmes de décision séquentielle les plus complexes : l'espace d'états atteint environ 2.08×10^{170} configurations distinctes, soit plusieurs ordres de grandeur au-delà des jeux d'échecs ou de dames. La difficulté principale réside dans l'impossibilité d'une recherche exhaustive ; des heuristiques d'évaluation de position et de sélection de coups performantes sont donc indispensables.

Depuis les travaux pionniers d'AlphaGo [1], il est établi que les réseaux de neurones convolutionnels surpassent largement les systèmes à base de règles pour ces deux sous-tâches. Ce projet explore dans quelle mesure des architectures modernes peuvent atteindre de bonnes performances de prédiction sous une contrainte sévère de capacité : **moins de 100 000 paramètres**.

Afin de garantir une comparaison équitable entre toutes les architectures, nous nous sommes imposé une contrainte académique stricte : **tous les hyperparamètres sont fixés et identiques** pour l'ensemble des modèles. Le taux d'apprentissage (`lr = 0.001`), le coefficient de régularisation L2 (`l2 = 1e-4`), le nombre maximal d'époques (300) et la graine aléatoire (`seed = 42`) sont constants. Seule l'architecture du réseau varie d'un modèle à l'autre. Ce protocole assure que les différences de performance observées sont imputables uniquement aux choix architecturaux, et non à un avantage obtenu par optimisation des hyperparamètres.

Objectifs :

- Évaluer dix familles architecturales classiques (MLP, CNN, ResNet, Transformer) dans un cadre à faible capacité avec les mêmes hyper-paramètres.
- Identifier les meilleures architectures et les renforcer avec un volume de données plus important.
- Tester des architectures efficientes modernes (MobileNet-style, ConvNeXt, EfficientFormer) spécialement conçues pour optimiser le rapport performance/paramètres.

Structure du rapport : La section 2 décrit le cadre expérimental commun aux trois phases. Les sections 3, 4 et 5 présentent les résultats de chaque phase. La section 6 synthétise le classement global, et la section 7 conclut avec des perspectives d'amélioration.

2 Cadre expérimental

2.1 Représentation de l'entrée

Chaque position de jeu de Go est représentée par un tenseur de forme $19 \times 19 \times 31$, soit 31 plans binaires encodant l'état courant du plateau. Ces plans incluent typiquement : la couleur des pierres de chaque joueur sur les 2 à 8 derniers coups, les libertés des groupes, les coups légaux et les coups kamikaze. Cette représentation multi-plan permet au réseau d'accéder à l'historique récent des positions sans mécanisme de mémoire externe.

2.2 Sorties et fonctions de perte

Le réseau possède deux têtes de sortie indépendantes :

- **Tête politique** : vecteur de 361 logits (un par intersection du plateau 19×19) suivi d'une activation `softmax`. La perte associée est l'entropie croisée catégorielle parcimonieuse (`sparse_categorical_crossentropy`).
- **Tête valeur** : scalaire en $[0, 1]$ suivi d'une activation `sigmoid`, représentant la probabilité de victoire du joueur courant. La perte associée est l'erreur quadratique moyenne (`MSE`).

La perte totale est la somme des deux pertes partielles, optimisée conjointement. La métrique principale retenue pour comparer les modèles est la **précision catégorielle de la politique en validation** (`val_policy_categorical_accuracy`), qui mesure le pourcentage de coups correctement prédits parmi les 361 possibles sur le jeu de validation fixe.

2.3 Protocole d'entraînement

Tous les modèles partagent la même infrastructure d'entraînement (Table 1). L'implémentation s'appuie sur Keras/TensorFlow 2 et s'exécute sur les GPU Kaggle (stratégie `MirroredStrategy` en multi-GPU).

Table 1 – Hyperparamètres communs à toutes les phases :

Hyperparamètre	**Valeur**
Optimiseur	Adam
Taux d'apprentissage initial	0,001
Taille de batch	$128 \times \#\text{GPU}$
Précision numérique	Mixed precision float16
Arrêt précoce (patience)	20 époques
ReduceLROnPlateau	facteur 0,5 – patience 10
Graine aléatoire	42 (`TF_DETERMINISTIC_OPS=1`)
Module de données	`golois` (jeu de données Kaggle)
Données de validation	Ensemble fixe (non modifié entre phases)

L'arrêt anticipé (*early stopping*) est déclenché lorsque la perte totale sur le jeu de validation (`val_loss`) ne s'améliore pas pendant 20 époques consécutives. Cette technique a été adoptée pour optimiser l'utilisation des ressources de calcul en ne poursuivant pas inutilement des runs qui ont convergé ou sont bloqués dans un plateau.

Au cours des entraînements, nous avons observé des instabilités numériques liées à l'explosion des gradients, en particulier lors des premières époques. Pour remédier à ce problème, nous avons appliqué un *gradient clipping* via le paramètre `clipnorm=1.0` de l'optimiseur Adam, qui normalise le gradient global dès que sa norme dépasse 1. Cette correction a permis de stabiliser la convergence et d'éviter les divergences de la fonction de perte.

Par ailleurs, nous avons délibérément écarté l'option d'augmenter le nombre d'époques au-delà de 300, estimant que le gain marginal espéré ne justifiait pas le coût computationnel supplémentaire.

2.4 Données d'entraînement

Le module `golois` fournit des parties de Go annotées par un moteur de force professionnelle. En Phase 1, l'entraînement utilise $N = 10\,000$ positions. En Phase 2, N est porté à $50\,000$ positions pour les trois meilleures architectures afin d'évaluer l'impact du volume de données. En Phase 3, les nouvelles architectures sont entraînées avec $N = 50\,000$ positions d'emblée.

2.5 Environnement de calcul et outils

Infrastructure de calcul :

L'entraînement des modèles a nécessité plusieurs environnements de calcul, chacun présentant des avantages et des contraintes spécifiques.

Google Colab (Pro) a été la première plateforme utilisée, donnant accès à des GPU L4 et A100. Cependant, cette solution s'est révélée peu adaptée aux entraînements longs : la session exige que l'ordinateur reste connecté en permanence, et des crashs noyau intempestifs interrompaient parfois les runs en cours d'exécution.

Juliet Mésonet (cluster de calcul du LAMSADE) a ensuite été utilisée. Si cette infrastructure offre des ressources plus stables, elle impose un système de file d'attente partagée : les temps d'attente pouvaient être significatifs selon la charge des machines par les autres utilisateurs.

Kaggle Notebooks a finalement constitué la solution retenue pour la majorité des entraînements. L'environnement cloud permet de lancer une exécution et de la laisser tourner de manière entièrement autonome, sans nécessiter de connexion active. Deux configurations matérielles ont été exploitées : deux GPU T4 en parallèle et un GPU P100, pour un quota hebdomadaire de 30 heures de calcul gratuit.

Suivi des expériences :

Deux outils complémentaires ont été utilisés pour le suivi des métriques d'entraînement. **TensorBoard** a permis une visualisation locale des courbes de perte et de précision. **Weights & Biases (WandB)** a assuré un suivi centralisé et persistant dans le cloud : chaque run enregistrait automatiquement les métriques epoch par epoch (perte totale, précision politique, MAE valeur) ainsi que les hyperparamètres associés, facilitant la comparaison entre architectures.

Sauvegarde et synchronisation

L'ensemble des fichiers produits lors de l'entraînement (checkpoints `.h5 et .keras`, courbes intermédiaires, fichiers CSV d'historique) était synchronisé automatiquement vers **Google Drive** à la fin de chaque run, garantissant la persistance des résultats entre les sessions d'entraînement.

3 Phase 1 : Exploration de dix architectures

3.1 Architectures explorées

La première phase explore dix familles architecturales couvrant un spectre large, du plus simple au plus sophistiqué.

M1 – MLP Baseline : Réseau entièrement connecté : l'entrée $19 \times 19 \times 31 = 11\,191$ valeurs est aplatie puis projetée via deux couches denses. Ce modèle ne possède aucun biais inductif spatial et sert de borne inférieure de référence.

M2 – CNN Shallow : Trois couches convolutionnelles `Conv2D(32, 3×3)` avec activation ReLU, sans normalisation par lot. Architecture CNN de référence minimale.

M3 – CNN + BatchNorm : CNN avec normalisation par lot (BN) après chaque convolution (Conv $\rightarrow$ BN $\rightarrow$ ReLU). La BN stabilise les gradients, permet des taux d'apprentissage plus élevés et réduit la sensibilité à l'initialisation.

M4 – ResNet Tiny : Blocs résiduels avec connexions skip. L'initialisation $\gamma = 0$ dans la dernière couche BN de chaque bloc (technique *zero-init residuals*) facilite l'entraînement en préservant l'identité au début.

M5 – Depthwise Separable CNN : Remplacement de chaque `Conv2D` par une convolution séparable en profondeur (`DepthwiseConv2D` + `Conv1×1`) pour maximiser l'efficacité paramétrique. Correspond à l'architecture MobileNetV1 [2]. La contrainte en paramètres entraîne ici une capacité insuffisante.

M6 – CNN Asymétrique : Têtes politique et valeur de capacités différentes : la tête politique reçoit davantage de filtres et de profondeur. Teste si l'asymétrie des têtes aide à spécialiser les représentations.

M7 – MobileNet-style : Blocs d'inversement résiduel (inverted residuals) avec taux d'expansion $\times 6$, comme MobileNetV2 [3]. La structure "bottleneck" applique la convolution profonde dans un espace de grande dimension puis projette dans un espace de petite dimension.

M8 – ResNet + SE : Blocs résiduels enrichis par des modules Squeeze-and-Excitation (SE) [4]. Le module SE recalibrate les activations canal par canal via un goulot d'étranglement global (*global average pooling* $\rightarrow$ Dense $\rightarrow$ Dense $\rightarrow$ Sigmoid).

M9 – ResNet + Dilated Conv : Convolutions dilatées avec taux $[1, 2, 3]$ dans les blocs résiduels. Le champ récepteur effectif atteint 21×21, couvrant presque entièrement la grille 19×19 sans augmentation du nombre de paramètres.

M10 – CNN + Transformer : Backbone CNN suivi d'un mécanisme d'attention multi-têtes (MHA, 4 têtes) sur les 19×19 tokens spatiaux, avec encodage positionnel appris. Le modèle hybride teste si l'attention globale apporte un gain sous la contrainte de 100k paramètres.

Table 2 – Récapitulatif des dix architectures de la Phase 1

Modèle	**Architecture**	**Famille**	**Params**	**Idée principale**
M1	MLP Baseline	MLP	99 904	Aucun biais inductif spatial
M2	CNN Shallow	CNN	97 388	3 convolutions sans normalisation
M3	CNN + BatchNorm	CNN	99 752	Conv $\rightarrow$ BN $\rightarrow$ ReLU
M4	ResNet Tiny	ResNet	99 628	Connexions résiduelles, γ-init=0
M5	Depthwise Separable CNN	Séparable	99 752	Convolutions séparables en profondeur
M6	CNN Asymétrique	CNN	99 752	Capacité asymétrique politique/valeur
M7	MobileNet-style	Inversé	99 752	Inverted residuals, expansion $\times 6$
M8	ResNet + SE	ResNet+Att	99 628	Attention par canal (SE)
M9	ResNet + Dilated	ResNet	99 628	Dilatation [1,2,3], champ 21×21
M10	CNN + Transformer	Hybride	99 498	MHA(4 têtes) sur tokens 19×19

3.2 Résultats de la Phase 1

La Table 3 présente les résultats de validation pour les dix modèles, avec la meilleure précision politique atteinte et l'époque à laquelle l'entraînement s'est arrêté (arrêt précoce ou saturation).

Table 3 – Résultats de validation – Phase 1 ($N = 10\,000$ positions).

Modèle	Architecture	Meilleure acc.	Époque	Arrêt
M9	ResNet + Dilated	**32,71 %**	230	Early stop
M4	ResNet Tiny	31,08 %	180	Early stop
M3	CNN + BatchNorm	29,87 %	110	Early stop
M7	MobileNet-style	12,19 %	50	Early stop
M8	ResNet + SE	11,41 %	50	Early stop
M2	CNN Shallow	11,12 %	200	Early stop
M6	CNN Asymétrique	6,13 %	100	Early stop
M1	MLP Baseline	0,57 %	140	Non (140 ép.)
M5	Depthwise Sep. CNN	0,39 %	–	Saturation ép. 40
M10	CNN + Transformer	0,39 %	–	Saturation ép. 60

3.3 Courbes d'entraînement

La Figure 1, la Figure 2 et la Figure 3 illustrent trois comportements contrastés observés en Phase 1 : la saturation complète de M5, la convergence progressive de M3, et les meilleures performances de M9.

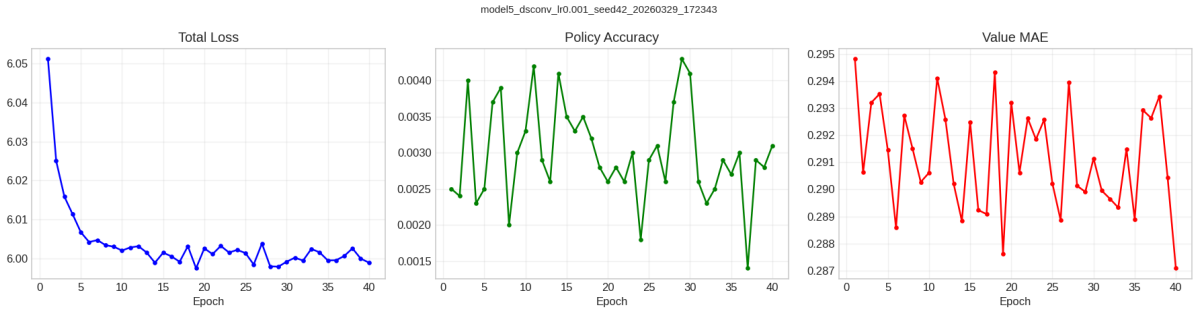

Figure 1 – M5 – Depthwise Sep. CNN (saturation dès l'époque 40)

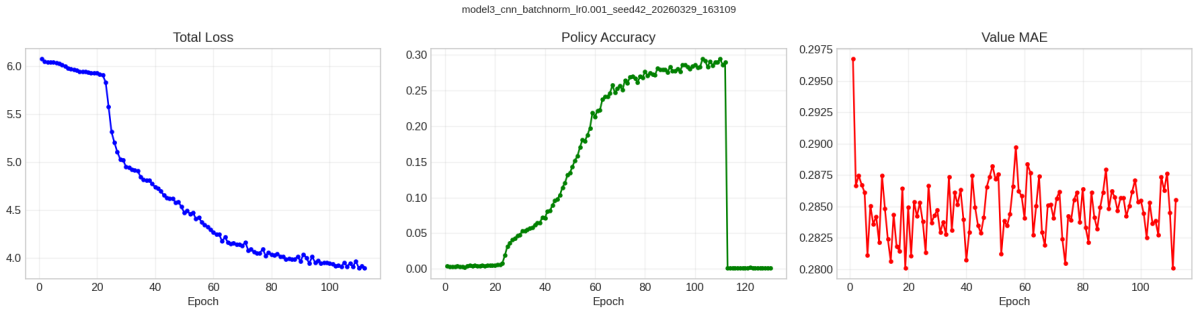

Figure 2 – M3 – CNN + BatchNorm (convergence régulière, 29,87 %)

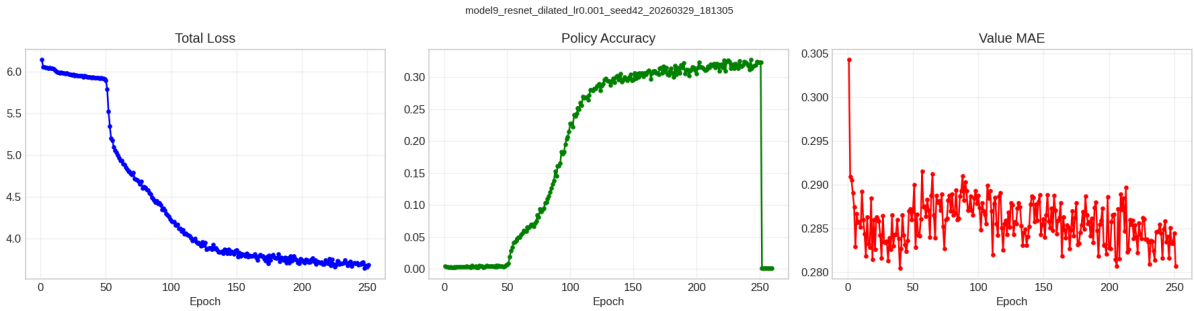

FIGURE 3 – M9 – ResNet + Dilated (meilleur modèle Phase 1, 32,71 %)

3.4 Analyse

Modèles en échec (M1, M5, M10) : Trois modèles n'apprennent pratiquement rien ($\leq 0,6\,\%$) :

- **M1 (MLP)** : sans biais inductif spatial, le réseau ne parvient pas à extraire les relations locales essentielles au jeu de Go. L'aplatissement de l'entrée détruit toute structure 2D.
- **M5 (Depthwise Sep.)** : sous la contrainte de 100k paramètres, la capacité résiduelle après séparation en profondeur est insuffisante pour la complexité de la tâche. Le modèle sature dès l'époque 40 avec des gradients quasiment nuls.
- **M10 (CNN+Transformer)** : le mécanisme d'attention MHA sur 19×19 tokens requiert un grand nombre de têtes ou de dimensions pour être efficace. Sous 100k paramètres, la portion allouée à l'attention est trop limitée et la saturation survient dès l'époque 60.

Rôle de la normalisation par lot : La comparaison M2 (11,12 %) vs M3 (29,87 %) démontre l'impact décisif de la BatchNorm : à architecture identique, l'ajout de normalisation après chaque convolution multiplie par 2,7 la précision finale. La BN permet des gradients mieux conditionnés et une convergence plus rapide.

Avantage des convolutions dilatées (M9) : M9 se distingue nettement avec 32,71 %, dépassant M4 de 1,6 points. L'utilisation de taux de dilatation $[1, 2, 3]$ permet d'élargir le champ récepteur jusqu'à 21×21 sans paramètres supplémentaires. Sur un plateau 19×19, cet avantage est structurel : chaque neurone peut *voir* la quasi-totalité du plateau dès les premières couches.

ResNet vs SE : M4 (ResNet, 31,08 %) surpasse M8 (ResNet+SE, 11,41 %) malgré l'attention par canal. Cet écart suggère qu'avec un budget aussi restreint, les paramètres consommés par les goulots SE réduisent trop la capacité convolutionnelle utile.

4 Phase 2 : Entraînement approfondi des meilleures architectures

4.1 Motivation et sélection

À l'issue de la Phase 1, trois architectures se distinguent nettement : M3 (29,87 %), M4 (31,08 %) et M9 (32,71 %). La différence de performance entre M9 et les modèles du milieu de tableau (M7 $\approx 12\,\%$, M8 $\approx 11\,\%$) est suffisamment importante pour justifier de ne pas retenir ces derniers.
L'hypothèse testée en Phase 2 est que le volume de données est un levier de performance majeur. Les expériences en apprentissage profond montrent systématiquement que les modèles sous-performent non pas à cause de leur architecture mais d'une quantité insuffisante de données d'entraînement. On passe donc de $N = 10\,000$ à $N = 50\,000$ positions, soit un facteur $\times 5$.

4.2 Résultats

TABLE 4 – Comparaison Phase 1 ($N = 10$k) vs Phase 2 ($N = 50$k) pour les trois meilleurs modèles.

Modèle	Architecture	Phase 1	Phase 2	Gain	Époque
M4	ResNet Tiny	31,08 %	**35,29 %**	+4,21 pp	170
M9	ResNet+Dilated	32,71 %	35,01 %	+2,30 pp	240
M3	CNN+BatchNorm	29,87 %	33,87 %	+4,00 pp	170

4.3 Courbes d'entraînement

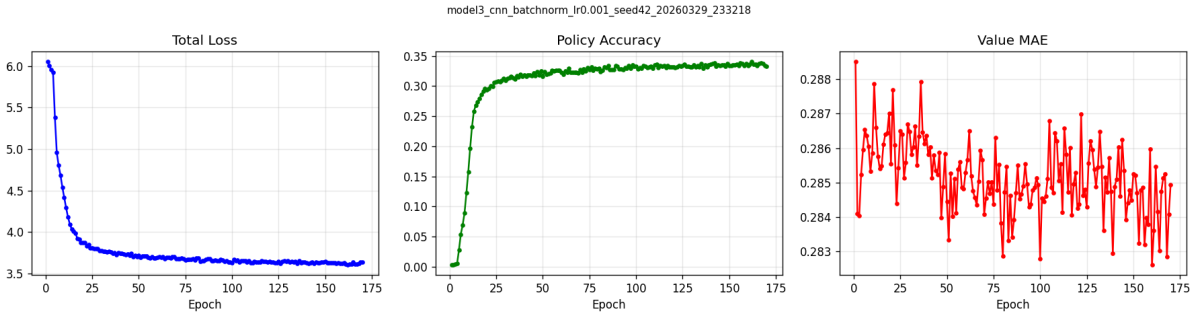

FIGURE 4 – M3 – CNN+BN (retrain) – 33,87 % à l'époque 170

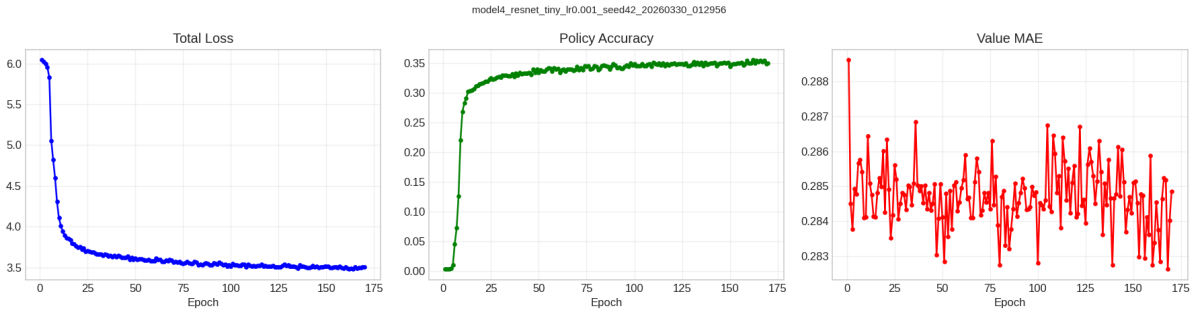

FIGURE 5 – M4 – ResNet Tiny (retrain) – 35,29 % à l'époque 170

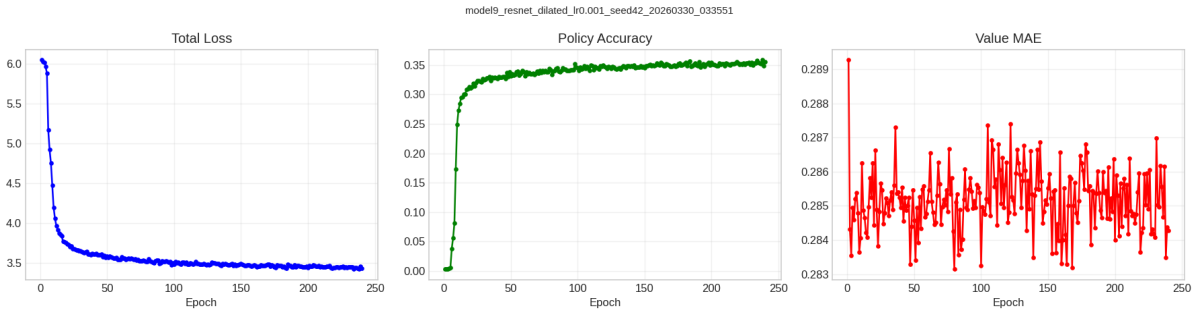

FIGURE 6 – M9 – ResNet+Dilated (retrain) – 35,01 % à l'époque 240

Courbes d'entraînement des trois modèles réentraînés avec $N = 50\,000$ positions (Phase 2). La convergence est plus régulière et les performances finales supérieures à la Phase 1.

4.4 Analyse

Impact du volume de données : Les trois modèles progressent de 2,3 à 4,2 points de pourcentage en multipliant le volume de données par 5. M4 enregistre le plus grand gain (+4,21 pp), ce qui suggère que les blocs résiduels bénéficient particulièrement de la diversité supplémentaire pour affiner les connexions skip. M9, déjà très performant en Phase 1 grâce à son champ récepteur élargi, progresse plus modestement (+2,30 pp) : son architecture exploite mieux les 10k exemples initiaux.

Inversion du classement : Fait notable, M4 dépasse M9 en Phase 2 (35,29 % vs 35,01 %), alors que M9 dominait en Phase 1. Avec plus de données, la capacité des blocs résiduels standards à apprendre des représentations générales comble l'avantage structurel des convolutions dilatées.

Limite de la Phase 2 : Malgré les progrès, les performances plafonnent autour de 35 %, loin des niveaux atteignables par des modèles sans contrainte. La section suivante explore des architectures modernes conçues pour maximiser l'efficacité sous budget paramétrique restreint.

5 Phase 3 : Architectures efficientes modernes

5.1 Motivation

Les résultats des Phases 1 et 2 révèlent que les architectures ResNet classiques atteignent un plafond autour de 35 % sous la contrainte de 100k paramètres. Deux facteurs limitants ont été identifiés :

1. **Inefficacité paramétrique des Conv2D standards** : une convolution 3×3 à C canaux d'entrée et C canaux de sortie coûte $9C^2$ paramètres. Les convolutions séparables en profondeur réduisent ce coût à $9C + C^2$ (facteur $\approx C/8$ d'économie pour C grand).
2. **Attention globale coûteuse** : le module SE [4] utilise deux couches denses de réduction $C \to C/r \to C$, coûtant $\approx 2C^2/r$ paramètres. L'attention ECA [5] obtient un effet similaire avec une convolution 1D de taille k (3 à 5 paramètres seulement).

La Phase 3 teste quatre architectures qui combinent ces techniques d'efficience avec des innovations récentes : activation Swish, noyaux 7×7 dépthwise (ConvNeXt), et hybridation CNN-Transformer à coût contrôlé.

5.2 Architectures de la Phase 3

M11 – MBConv + SE (98 242 paramètres) : Blocs MBConv (Mobile Inverted Bottleneck Convolution) avec expansion $\times 4$, convolution séparable 3×3, activation Swish et module SE. La technique de *stochastic depth* (abandon aléatoire de blocs durant l'entraînement) régularise l'entraînement. Inspiré d'EfficientNet [6].

M12 – Inverted Residual + ECA (93 773 paramètres) : Variante de M11 remplaçant le module SE par l'attention par canal efficiente (ECA) [5] : une convolution 1D de noyau adaptatif sur les activations globales, ne coûtant que 3 à 5 paramètres contre $\approx 1\,600$ pour un SE équivalent. Le budget économisé est réalloué aux couches convolutionnelles.

M15 – ConvNeXt-Lite (92 863 paramètres) : Adaptation de ConvNeXt [7] : noyaux dépthwise 7×7, normalisation LayerNorm (plutôt que BatchNorm), activation GELU, et expansion MLP ×4. Cette architecture "modernise" le CNN classique en incorporant les inductive biases des Transformeurs dans un cadre entièrement convolutionnel.

M18 – EfficientFormer-Go (99 666 paramètres) : Architecture étagée hybride CNN-Transformer inspirée d'EfficientFormer [8] : les premières couches sont convolutionnelles (extraction de caractéristiques locales) et les dernières appliquent une attention multi-têtes sur des tokens poolés, réduisant ainsi le coût quadratique de l'attention.

Table 5 – Caractéristiques des quatre architectures avancées (Phase 3).

Modèle	Architecture	Blocs principaux	Attention	Params
M11	MBConv + SE	MBConv × 3, exp.×4	SE	98 242
M12	Inverted + ECA	InvRes × 3, exp.×4	ECA (≤5 param.)	93 773
M15	ConvNeXt-Lite	ConvNeXt × 2, $k = 7$, MLP×4	–	92 863
M18	EfficientFormer	CNN → MHA étagé	MHA poolé	99 666

5.3 Résultats

Les performances de la Phase 3 sont exportées depuis WandB. Les métriques sont enregistrées à chaque étape de validation (une étape ≈ une époque, avec $N = 50\,000$ positions).

Table 6 – Résultats de validation – Phase 3 (WandB exports).

Modèle	Architecture	Acc. max	Étape du pic	Étapes totales
M18	EfficientFormer	**44,56 %**	268	300
M11	MBConv + SE	44,39 %	230	238
M12	Inverted + ECA	43,11 %	186	190
M15	ConvNeXt-Lite	41,19 %	141	142 (encore en hausse)

5.4 Analyse

Saut de performance remarquable : Les quatre architectures de la Phase 3 dépassent toutes les performances de la Phase 2 (plafond à 35,29 %). Le meilleur modèle, M18, atteint 44,56 %, soit un gain de **+9,3 points** par rapport à M4 retrained. Ce saut illustre l'avantage structurel des architectures efficientes modernes : elles parviennent à extraire davantage d'information avec le même budget paramétrique.

M18 vs M11 : hybridation CNN-Transformer : M18 (44,56 %) dépasse légèrement M11 (44,39 %). L'avantage de l'hybridation CNN-Transformer étagée réside dans la capacité à capturer des dépendances à longue portée (attention globale) tout en conservant des biais inductifs spatiaux locaux (couches convolutionnelles initiales). Cependant, l'écart est faible (0,17 pp), suggérant que les deux approches convergent vers un optimum similaire sous 100k paramètres.

M12 : ECA comme alternative viable au SE : M12 (43,11 %) confirme que l'attention ECA est une alternative crédible au SE classique. Avec moins de 5 paramètres pour le module d'attention (contre ≈1 600 pour SE), l'ECA libère de la capacité pour les couches convolutionnelles

tout en maintenant un mécanisme de recalibration efficace. L'écart de 1,3 pp entre M11 et M12 est donc principalement imputable à l'efficacité de SE sur ce domaine.

M15 : ConvNeXt non convergé : M15 (41,19 %) est encore en hausse à l'étape 141 (dernière étape du run). ConvNeXt, avec ses noyaux 7×7 et son expansion MLP $\times 4$, requiert davantage d'itérations pour ajuster ses nombreuses interactions à longue portée. Il est raisonnable de supposer que M15 aurait rejoint les performances de M11/M18 avec 100 étapes supplémentaires.

6 Comparaison globale

La Table 7 classe tous les modèles évalués par précision politique décroissante, toutes phases confondues.

Table 7 – Classement global de toutes les architectures évaluées.

Rang	Modèle	Phase	Données	Acc. max
1	M18 – EfficientFormer	3	50k	**44,56 %**
2	M11 – MBConv + SE	3	50k	44,39 %
3	M12 – Inverted + ECA	3	50k	43,11 %
4	M15 – ConvNeXt-Lite	3	50k	41,19 %†
5	M4 – ResNet Tiny	2 (retrain)	50k	35,29 %
6	M9 – ResNet + Dilated	2 (retrain)	50k	35,01 %
7	M3 – CNN + BatchNorm	2 (retrain)	50k	33,87 %
8	M9 – ResNet + Dilated	1	10k	32,71 %
9	M4 – ResNet Tiny	1	10k	31,08 %
10	M3 – CNN + BatchNorm	1	10k	29,87 %
11	M7 – MobileNet-style	1	10k	12,19 %
12	M8 – ResNet + SE	1	10k	11,41 %
13	M2 – CNN Shallow	1	10k	11,12 %
14	M6 – CNN Asymétrique	1	10k	6,13 %
15	M1 – MLP Baseline	1	10k	0,57 %
16	M5 – Depthwise Sep. CNN	1	10k	0,39 %
16	M10 – CNN + Transformer	1	10k	0,39 %

† M15 n'avait pas convergé à la fin du run (courbe encore croissante).

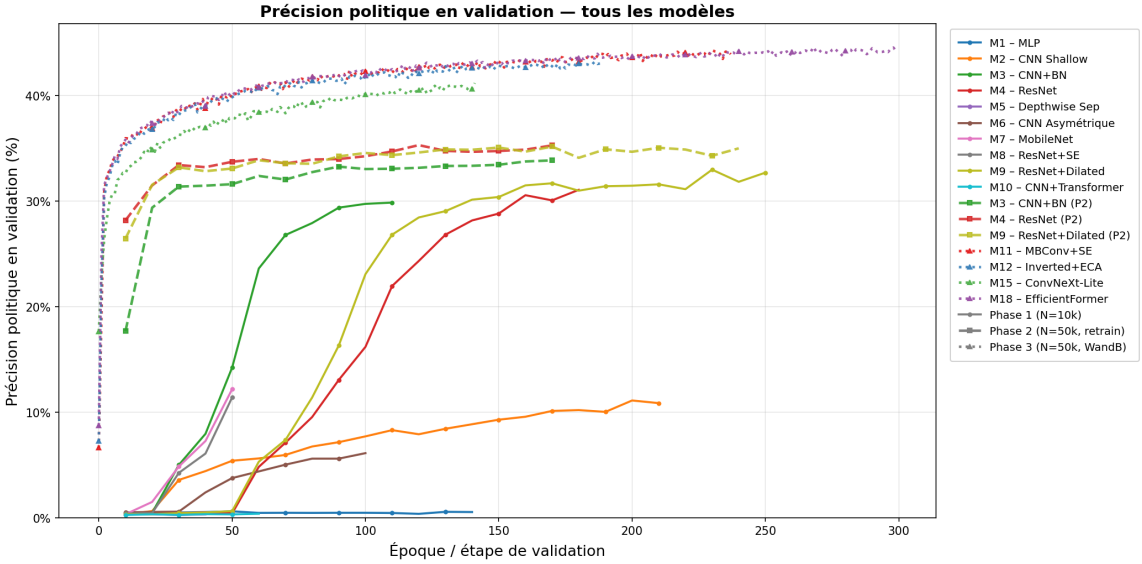

FIGURE 7 – Évolution de la précision politique en validation pour l'ensemble des modèles et des trois phases d'entraînement. Les courbes en trait plein correspondent à la Phase 1 ($N = 10\,000$), les tiretés à la Phase 2 ($N = 50\,000$, réentraînement des top 3), et les pointillés à la Phase 3 ($N = 50\,000$, nouvelles architectures MobileNet-style). La Phase 3 domine nettement, M18 (EfficientFormer) atteignant 44,56 % à l'époque 300.

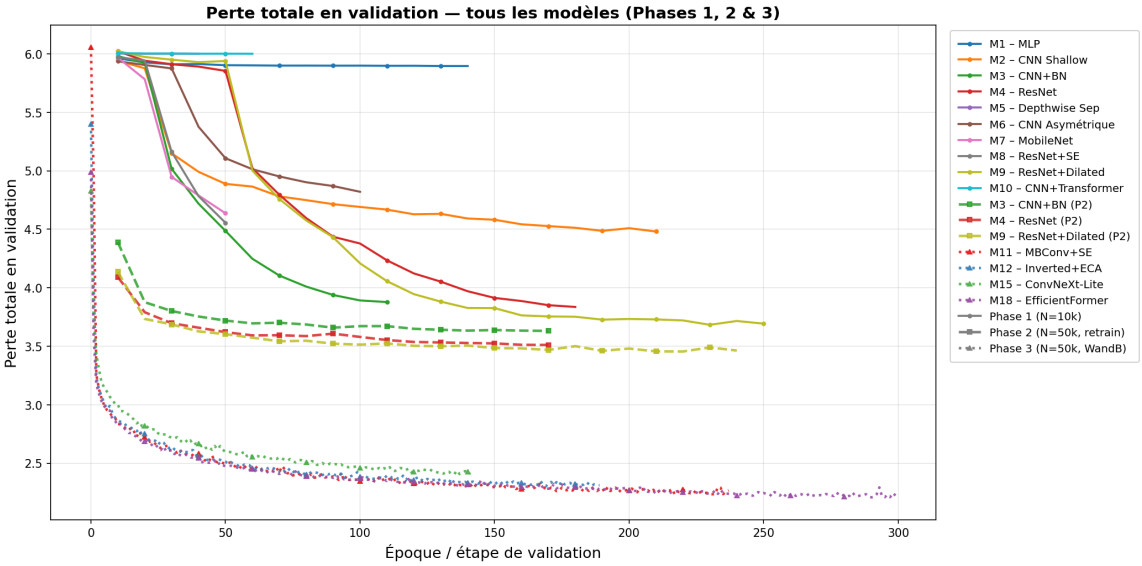

FIGURE 8 – Évolution de la perte totale en validation pour les Phases 1, 2 et 3. M1 (MLP) et M10 (CNN+Transformer) ne convergent pas (perte stagnante autour de 5,9–6,0). Les modèles de Phase 3 atteignent les pertes les plus basses (autour de 2,5), tandis que les modèles de Phase 2 se stabilisent autour de 3,5 (M9-P2), confirmant la hiérarchie observée en précision politique.

6.1 Discussion

Biais inductif spatial (condition nécessaire) : Les modèles sans biais inductif spatial approprié (M1 MLP : 0,57 %) ou avec un budget trop contraint pour exprimer leur biais (M5 séparable : 0,39 %) échouent systématiquement. En revanche, dès qu'une structure convolutionnelle est présente avec une normalisation adéquate (M3 : 29,87 %), les performances décollent. Ce résultat confirme que l'inductive bias spatial des CNN est fondamental pour la prédiction dans le

Go.

Volume de données (levier secondaire mais significatif) : Le passage de 10k à 50k positions apporte un gain systématique de 2 à 4 points sur les mêmes architectures. Ce gain est réel mais insuffisant à lui seul : les architectures ResNet classiques plafonnent autour de 35 % quelle que soit la quantité de données testée.

Architecture efficiente (levier primaire) : Le saut le plus important provient du choix architectural : passer de ResNet à MBConv/ConvNeXt/EfficientFormer apporte +9 points supplémentaires à N identique. Les techniques modernes (blocs inverted residual, attention légère, activation Swish/GELU, noyaux larges dépthwise) maximisent le rapport information/paramètre de manière incomparablement plus efficace que les blocs résiduels classiques.

Attention globale sous contrainte : L'attention multi-têtes standard (M10, Phase 1) échoue complètement sous 100k paramètres. En revanche, des formes légères d'attention globale – SE, ECA, MHA poolé (M18) – apportent toutes un gain net. La clé est de limiter le coût de l'attention pour préserver la capacité convolutionnelle.

7 Conclusion

Ce projet a évalué quatorze architectures de réseaux de neurones profonds pour la prédiction de coups et l'évaluation de position dans le jeu de Go 19×19, sous une contrainte stricte de 100 000 paramètres et sous la contrainte académique d'uniformité des hyperparamètres entre tous les modèles.

Synthèse : Trois conclusions principales se dégagent :

1. **Le biais inductif spatial est indispensable** : MLP et CNN mal configurés n'apprennent rien de significatif.
2. **Le volume de données amplifie les bonnes architectures** : passer de 10k à 50k positions apporte +2 à +4 points selon le modèle.
3. **Les architectures efficientes modernes dominent** : les blocs MBConv, l'attention légère (SE, ECA) et les hybrides CNN-Transformer surpassent les ResNet classiques de 9 points en utilisant le même budget paramétrique.

Meilleur modèle : M18 (EfficientFormer-Go, 99 666 paramètres) atteint **44,56 %** de précision politique en validation, suivi de très près par M11 (MBConv+SE, 44,39 %). Ces deux modèles représentent l'état de l'art atteignable sous la contrainte imposée avec le protocole d'entraînement utilisé.

Perspectives : Plusieurs axes d'amélioration sont envisageables :

- **Augmentation de données** : symétries du plateau de Go (rotations 90°, réflexions) ×8 permettraient d'augmenter artificiellement le corpus.
- **Scheduler cosinus** : remplacer ReduceLROnPlateau par un cosine annealing améliore la convergence des architectures modernes.
- **Architecture Search (NAS)** : une recherche automatique d'architecture sous contrainte de 100k paramètres pourrait identifier des configurations encore plus efficaces.
- **Distillation** : entraîner un petit modèle (<100k params) en imitant un grand modèle sans contrainte (distillation de connaissances).

Références

[1] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, et D. Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587) :484–489, 2016.

[2] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, et H. Adam. MobileNets : Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv :1704.04861*, 2017.

[3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, et L.-C. Chen. MobileNetV2 : Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018.

[4] J. Hu, L. Shen, et G. Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7132–7141, 2018.

[5] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, et Q. Hu. ECA-Net : Efficient channel attention for deep convolutional neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11534–11542, 2020.

[6] M. Tan et Q. V. Le. EfficientNet : Rethinking model scaling for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 6105–6114, 2019.

[7] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, et S. Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11976–11986, 2022.

[8] Y. Li, G. Yuan, Y. Wen, J. Hu, G. Evangelidis, S. Tulyakov, Y. Wang, et J. Ren. EfficientFormer : Vision transformers at MobileNet speed. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 12934–12949, 2022.